BSAN 360 Final Project

Item 2 • PDF Copy of Python Notebook

Eduardo Maduro

Compiled notebook export for final submission

This file combines the notebook PDF exports from the project milestones into one submission-ready PDF package.

Included sections:

- Project 2: data cleaning and preparation
- Project 3: low-sample filtering, reshaping, pivots, and saved cleaned file
- Project 4: exploratory data analysis and visualizations
- Project 5: transformations, grouping, regression, and written explanation
- Final project notebook: regression, ANOVA, and final results comment

Note: this PDF is assembled from the uploaded Colab PDF exports you provided. It is suitable for the PDF submission requirement, though it is not the editable .ipynb notebook file itself.

Prepared on March 16, 2026

```
# ============================================================
# PROJECT CONTINUATION — WEEK 6
# Data Cleaning and Preparation
# Name: Eduardo Maduro
# Course: BSAN 360
# ============================================================
```

```
from google.colab import files
uploaded = files.upload()
```

Choose Files all_seasons.csv

all_seasons.csv(text/csv) - 1920482 bytes, last modified: 10/13/2023 - 100% done

```
Saving all_seasons.csv to all_seasons.csv
```

```
import os
os.listdir("/content")
```

```
['.config', 'all_seasons.csv', 'sample_data']
```

```
import pandas as pd

df = pd.read_csv("all_seasons.csv")
df.head()
```

	Unnamed: 0	player_name	team_abbreviation	age	player_height	player_weight	college	country	draft_year	draft_
0	0	Randy Livingston	HOU	22.0	193.04	94.800728	Louisiana State	USA	1996	
1	1	Gaylon Nickerson	WAS	28.0	190.50	86.182480	Northwestern Oklahoma	USA	1994	
2	2	George Lynch	VAN	26.0	203.20	103.418976	North Carolina	USA	1993	
		George								

```
# ------------------------------------------------------------
# Dataset Overview
# ------------------------------------------------------------

df.info()
df.describe()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 12844 entries, 0 to 12843
Data columns (total 22 columns):
 #   Column             Non-Null Count  Dtype
---  ------             --------------  -----
 0   Unnamed: 0         12844 non-null  int64
 1   player_name        12844 non-null  object
 2   team_abbreviation  12844 non-null  object
 3   age                12844 non-null  float64
 4   player_height      12844 non-null  float64
 5   player_weight      12844 non-null  float64
 6   college            10990 non-null  object
 7   country            12844 non-null  object
 8   draft_year         12844 non-null  object
 9   draft_round        12844 non-null  object
 10  draft_number       12844 non-null  object
 11  gp                 12844 non-null  int64
 12  pts                12844 non-null  float64
 13  reb                12844 non-null  float64
```

```
# ------------------------------------------------------------
# Select relevant columns for analysis
# ------------------------------------------------------------

relevant_columns = [
    "player_name",
    "team_abbreviation",
    "age",
    "player_height",
    "player_weight",
    "country",
    "draft_year",
    "pts",
    "reb",
    "ast",
    "season"
]

df = df[relevant_columns]
df.head()
```

	player_name	team_abbreviation	age	player_height	player_weight	country	draft_year	pts	reb	ast	season
max	12843.000000	44.000000	231.140000	163.293120	85.000000	36.100000	16.300000	11.700000	30		
0	Randy Livingston	HOU	22.0	193.04	94.800728	USA	1996	3.9	1.5	2.4	1996-97
1	Gaylon Nickerson	WAS	28.0	190.50	86.182480	USA	1994	3.8	1.3	0.3	1996-97
2	George Lynch	VAN	26.0	203.20	103.418976	USA	1993	8.3	6.4	1.9	1996-97
3	George	[illegible]	[illegible]	[illegible]	[illegible]	USA	[illegible]	[illegible]	[illegible]	[illegible]	1996-

Next steps: Generate code with df New interactive sheet

```
# ------------------------------------------------------------
# Missing Data Analysis
# ------------------------------------------------------------

df.isnull().sum()
```

	0
player_name	0
team_abbreviation	0
age	0
player_height	0
player_weight	0
country	0
draft_year	0
pts	0
reb	0
ast	0
season	0

dtype: int64

```
# Note: After filtering to relevant columns, no missing values remain.
# This step is included to demonstrate proper handling if missing
# performance metrics were present.
```

```
# ------------------------------------------------------------
# Data Transformations
# ------------------------------------------------------------

df["season_start"] = df["season"].str[:4].astype(int)
```

```
# Create a total contribution metric
df["total_contribution"] = df["pts"] + df["reb"] + df["ast"]
```

```
# Derived variables help summarize player impact
# and simplify trend analysis across seasons.
```

```
df.info()
df.head()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 12844 entries, 0 to 12843
Data columns (total 13 columns):
 #   Column              Non-Null Count  Dtype
---  ------              --------------  -----
 0   player_name         12844 non-null  object
 1   team_abbreviation   12844 non-null  object
 2   age                 12844 non-null  float64
 3   player_height       12844 non-null  float64
 4   player_weight       12844 non-null  float64
 5   country             12844 non-null  object
 6   draft_year          12844 non-null  object
 7   pts                 12844 non-null  float64
```

```
from google.colab import files
uploaded = files.upload()
```

Choose Files all_seasons.csv

all_seasons.csv(text/csv) - 1920482 bytes, last modified: 10/13/2023 - 100% done

```
Saving all_seasons.csv to all_seasons.csv
```

```
import pandas as pd

# Load the dataset
df = pd.read_csv("all_seasons.csv")

# Check it loaded
df.head()
```

	Unnamed: 0	player_name	team_abbreviation	age	player_height	player_weight	college	country	draft_year	draft_
0	0	Randy Livingston	HOU	22.0	193.04	94.800728	Louisiana State	USA	1996	
1	1	Gaylon Nickerson	WAS	28.0	190.50	86.182480	Northwestern Oklahoma	USA	1994	
2	2	George Lynch	VAN	26.0	203.20	103.418976	North Carolina	USA	1993	
3	3	George McCloud	LAL	30.0	203.20	102.058200	Florida State	USA	1989	
4	4	George Zidek	DEN	23.0	213.36	119.748288	UCLA	USA	1995	

5 rows × 22 columns

```
import pandas as pd
import numpy as np

df = pd.read_csv("all_seasons.csv")

# Clean column names
df.columns = df.columns.str.strip().str.lower().str.replace(" ", "_")

# Clean string columns
for col in ["player_name", "team_abbreviation", "pos", "college", "country", "season"]:
    if col in df.columns:
        df[col] = df[col].astype(str).str.strip()

# Show columns so you can see what you have
print("Columns:", df.columns.tolist())
print("Shape:", df.shape)
display(df.head())

# Detect core columns (this dataset typically uses these exact names)
gp_col  = "gp"  if "gp"  in df.columns else None
min_col = "min" if "min" in df.columns else None   # minutes per game
pts_col = "pts" if "pts" in df.columns else None   # points per game
pos_col = "pos" if "pos" in df.columns else None
team_col = "team_abbreviation" if "team_abbreviation" in df.columns else None

print("\nDetected games column:", gp_col)
print("Detected minutes column:", min_col)
print("Detected points column:", pts_col)
print("Detected position column:", pos_col)
print("Detected team column:", team_col)

# ---- LOW SAMPLE FILTER (responds to professor comment) ----
# Use games played and/or minutes per game instead of points >= 100
df_clean = df.copy()

# Good default thresholds for this dataset
min_games_threshold = 20       # remove players with very few games
min_minutes_pg_threshold = 10  # remove players barely playing

if gp_col is not None:
```

```
    df_clean = df_clean[df_clean[gp_col] >= min_games_threshold]

if min_col is not None:
    df_clean = df_clean[df_clean[min_col] >= min_minutes_pg_threshold]

print("\nOriginal rows:", len(df))
print("Rows after low-sample filtering:", len(df_clean))

# ---- MERGE DEMO (even with one file) ----
# Create a small lookup table for teams and merge back in
if team_col is not None:
    team_lookup = df_clean[[team_col]].drop_duplicates().assign(team_clean=lambda x: x[team_col].str.upper().str.str
    df_merged = df_clean.merge(team_lookup, on=team_col, how="left")
    print("\nMerge completed using:", team_col)
else:
    df_merged = df_clean.copy()
    print("\nNo team column found; merge demo skipped.")

# ---- PIVOTS (reshaping) ----
# Pivot: Average PTS by position
if pts_col is not None and pos_col is not None:
    pivot_pos = (
        df_merged.pivot_table(values=pts_col, index=pos_col, aggfunc="mean")
        .sort_values(by=pts_col, ascending=False)
    )
    print("\nPivot: Average points per game by position")
    display(pivot_pos)
else:
    print("\nPivot by position skipped (missing pts or pos).")

# Pivot: Top 10 teams by average PTS
if pts_col is not None and team_col is not None:
    pivot_team = (
        df_merged.pivot_table(values=pts_col, index=team_col, aggfunc="mean")
        .sort_values(by=pts_col, ascending=False)
        .head(10)
    )
    print("\nPivot: Top 10 teams by average points per game")
    display(pivot_team)
else:
    print("\nPivot by team skipped (missing pts or team).")

# Save cleaned file
df_merged.to_csv("all_seasons_cleaned.csv", index=False)
print("\nSaved: all_seasons_cleaned.csv")
```

```
Columns: ['unnamed:_0', 'player_name', 'team_abbreviation', 'age', 'player_height', 'player_weight', 'college', 'cou
Shape: (12844, 22)
```

	unnamed:_0	player_name	team_abbreviation	age	player_height	player_weight	college	country	draft_year	dra
0	0	Randy Livingston	HOU	22.0	193.04	94.800728	Louisiana State	USA	1996	
1	1	Gaylon Nickerson	WAS	28.0	190.50	86.182480	Northwestern Oklahoma	USA	1994	
2	2	George Lynch	VAN	26.0	203.20	103.418976	North Carolina	USA	1993	
3	3	George McCloud	LAL	30.0	203.20	102.058200	Florida State	USA	1989	
4	4	George Zidek	DEN	23.0	213.36	119.748288	UCLA	USA	1995	

5 rows × 22 columns

```
Detected games column: gp
Detected minutes column: None
Detected points column: pts
Detected position column: None
Detected team column: team_abbreviation

Original rows: 12844
Rows after low-sample filtering: 10720

Merge completed using: team_abbreviation

Pivot by position skipped (missing pts or pos).

Pivot: Top 10 teams by average points per game
```

team_abbreviation	pts
NOP	10.370922
HOU	9.733523
PHX	9.686286
SAC	9.654261
GSW	9.638028
BKN	9.627044
OKC	9.569608
MIA	9.547429
LAL	9.534247
LAC	9.389008

```
Saved: all_seasons_cleaned.csv
```

Next steps: Generate code with pivot_team New interactive sheet

```
import pandas as pd
import numpy as np
import statsmodels.api as sm

df = pd.read_csv("all_seasons.csv")
df.columns = df.columns.str.strip().str.lower().str.replace(" ", "_")

# Drop useless index column if it exists
if "unnamed:_0" in df.columns:
    df = df.drop(columns=["unnamed:_0"])

print("Columns:", df.columns.tolist())
display(df.head())
```

```
Columns: ['player_name', 'team_abbreviation', 'age', 'player_height', 'player_weight', 'college', 'country', 'draft_y
```

	player_name	team_abbreviation	age	player_height	player_weight	college	country	draft_year	draft_round	dra
0	Randy Livingston	HOU	22.0	193.04	94.800728	Louisiana State	USA	1996	2	
1	Gaylon Nickerson	WAS	28.0	190.50	86.182480	Northwestern Oklahoma	USA	1994	2	
2	George Lynch	VAN	26.0	203.20	103.418976	North Carolina	USA	1993	1	
3	George McCloud	LAL	30.0	203.20	102.058200	Florida State	USA	1989	1	
4	George Zidek	DEN	23.0	213.36	119.748288	UCLA	USA	1995	1	

5 rows × 21 columns

```
# Filter players with very low games played
min_gp_threshold = 20
df_clean = df[df["gp"] >= min_gp_threshold].copy()

print("Original rows:", len(df))
print("Rows after gp filter:", len(df_clean))
```

```
Original rows: 12844
Rows after gp filter: 10720
```

```
import pandas as pd
import numpy as np
import statsmodels.api as sm

df = pd.read_csv("all_seasons.csv")
df.columns = df.columns.str.strip().str.lower().str.replace(" ", "_")
if "unnamed:_0" in df.columns:
    df = df.drop(columns=["unnamed:_0"])

# Low sample filter using games played
df = df[df["gp"] >= 20].copy()

# Keep columns we need
keep = ["pts","gp","age","player_height","player_weight","draft_year","draft_number","team_abbreviation"]
df_model = df[[c for c in keep if c in df.columns]].copy()

# Drop rows missing target
df_model = df_model.dropna(subset=["pts"])

# Make numeric columns numeric (coerce errors -> NaN)
num_cols = [c for c in ["pts","gp","age","player_height","player_weight","draft_year","draft_number"] if c in df_mode
for c in num_cols:
    df_model[c] = pd.to_numeric(df_model[c], errors="coerce")

# Drop rows with missing numeric predictors needed for each model later
# Team stays categorical
df_model = df_model.dropna(subset=["gp","age","player_height","player_weight","draft_year","draft_number"])

y = df_model["pts"]

# ----- Model 1 -----
X1 = sm.add_constant(df_model[["gp"]])
m1 = sm.OLS(y, X1).fit()

# ----- Model 2 -----
X2 = sm.add_constant(df_model[["gp","age","player_height","player_weight"]])
m2 = sm.OLS(y, X2).fit()

# ----- Model 3 (team dummies + draft info) -----
df_dum = pd.get_dummies(df_model, columns=["team_abbreviation"], drop_first=True)

X3_cols = ["gp","age","player_height","player_weight","draft_year","draft_number"]
team_cols = [c for c in df_dum.columns if c.startswith("team_abbreviation_")]

X3 = sm.add_constant(df_dum[X3_cols + team_cols])

# Force ALL columns to float (prevents object dtype errors)
```

```
X3 = X3.astype(float)

m3 = sm.OLS(y, X3).fit()

print("Model 1 R2:", round(m1.rsquared, 4), "Adj R2:", round(m1.rsquared_adj, 4))
print("Model 2 R2:", round(m2.rsquared, 4), "Adj R2:", round(m2.rsquared_adj, 4))
print("Model 3 R2:", round(m3.rsquared, 4), "Adj R2:", round(m3.rsquared_adj, 4))
```

```
Model 1 R2: 0.1673 Adj R2: 0.1672
Model 2 R2: 0.1752 Adj R2: 0.1748
```

```
# ==============================
# STEP 1: Upload and Load Data
# ==============================

from google.colab import files
uploaded = files.upload()

import pandas as pd

df = pd.read_csv('all_seasons.csv')

df.head()
```

Choose Files all_seasons.csv

all_seasons.csv(text/csv) - 1920482 bytes, last modified: 10/13/2023 - 100% done

```
Saving all_seasons.csv to all_seasons (1).csv
```

	Unnamed: 0	player_name	team_abbreviation	age	player_height	player_weight	college	country	draft_year	draft_
0	0	Randy Livingston	HOU	22.0	193.04	94.800728	Louisiana State	USA	1996	
1	1	Gaylon Nickerson	WAS	28.0	190.50	86.182480	Northwestern Oklahoma	USA	1994	
2	2	George Lynch	VAN	26.0	203.20	103.418976	North Carolina	USA	1993	
		George								

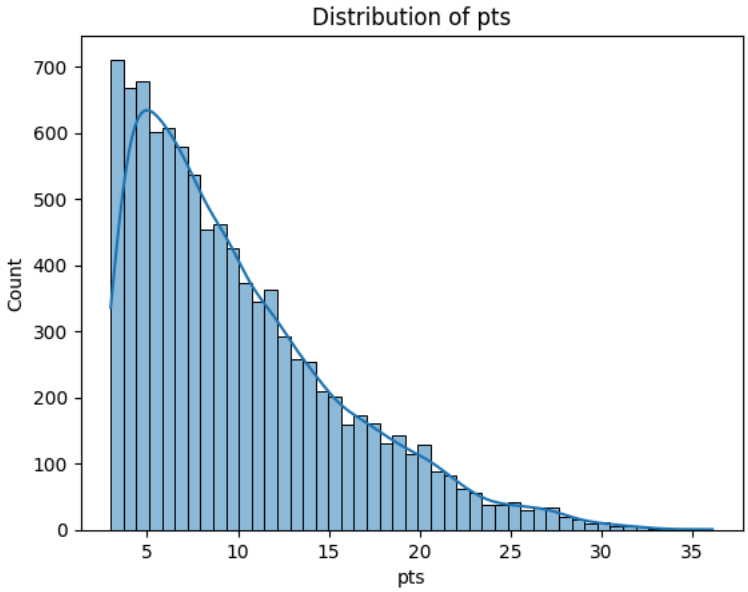
```
# Remove players with very low games played or very low scoring

df = df[(df['gp'] >= 20) & (df['pts'] >= 3)]

print("Remaining players:", df.shape[0])
```

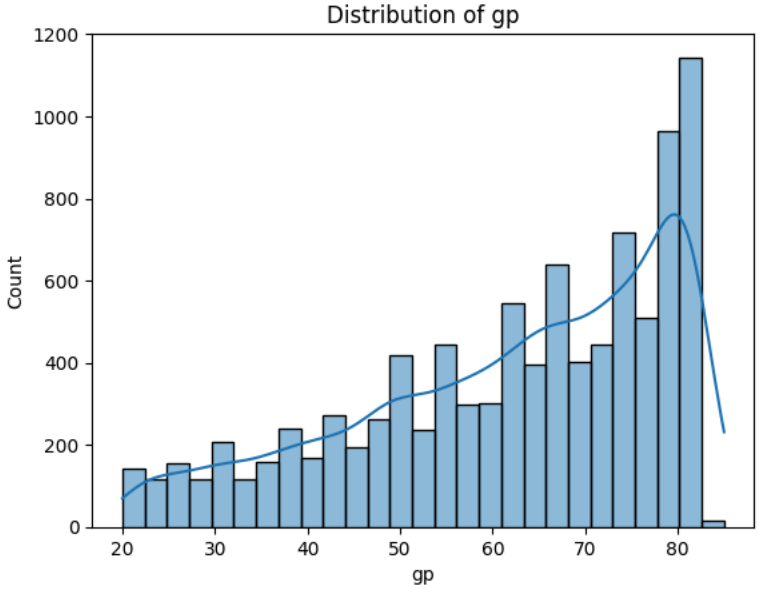
```
Remaining players: 9614
```

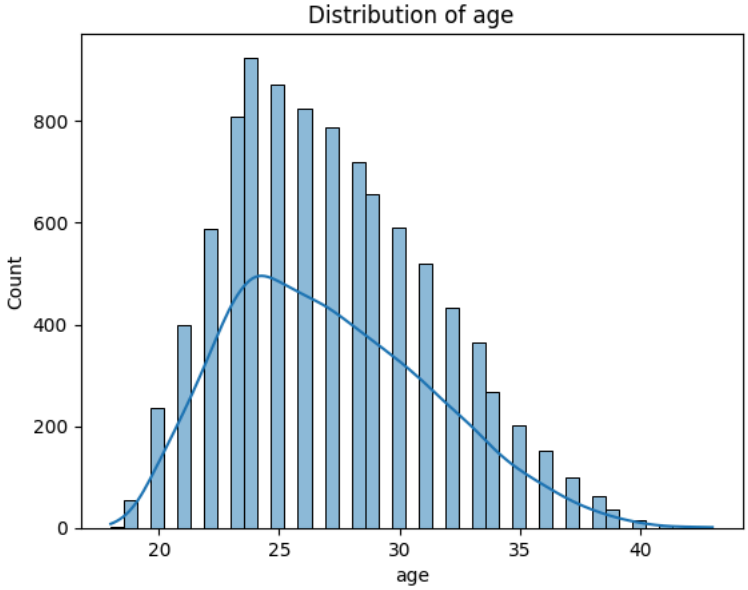
```
df.describe()
```

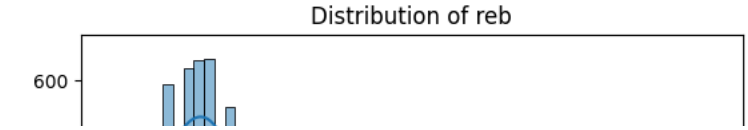
	Unnamed: 0	age	player_height	player_weight	gp	pts	reb	ast	net_rating
count	9614.000000	9614.000000	9614.000000	9614.000000	9614.000000	9614.000000	9614.000000	9614.000000	9614.000000
mean	6431.632411	27.157375	200.374701	100.135506	61.907011	10.039286	4.189422	2.210568	-0.714552
std	3682.813848	4.277351	9.077928	12.333698	16.777916	5.713810	2.442352	1.869191	6.099281
min	0.000000	18.000000	160.020000	60.327736	20.000000	3.000000	0.400000	0.000000	-27.700000
25%	3255.250000	24.000000	193.040000	90.718400	50.000000	5.600000	2.400000	0.900000	-4.900000
50%	6433.500000	27.000000	200.660000	99.790240	66.000000	8.600000	3.600000	1.600000	-0.500000
75%	9576.750000	30.000000	208.280000	108.862080	76.000000	13.200000	5.300000	2.900000	3.400000
max	12843.000000	43.000000	231.140000	155.582056	85.000000	36.100000	16.100000	11.700000	19.500000

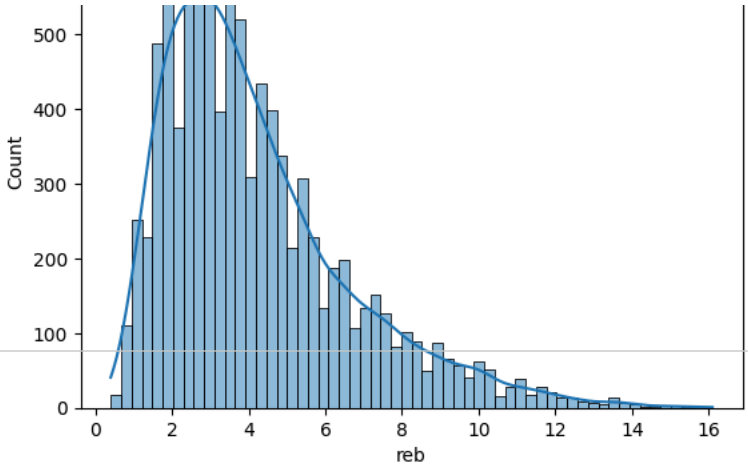
```
import matplotlib.pyplot as plt
import seaborn as sns

numeric_cols = ['pts', 'gp', 'age', 'reb', 'ast', 'usg_pct', 'ts_pct']

for col in numeric_cols:
    plt.figure()
    sns.histplot(df[col], kde=True)
    plt.title(f'Distribution of {col}')
    plt.show()
```

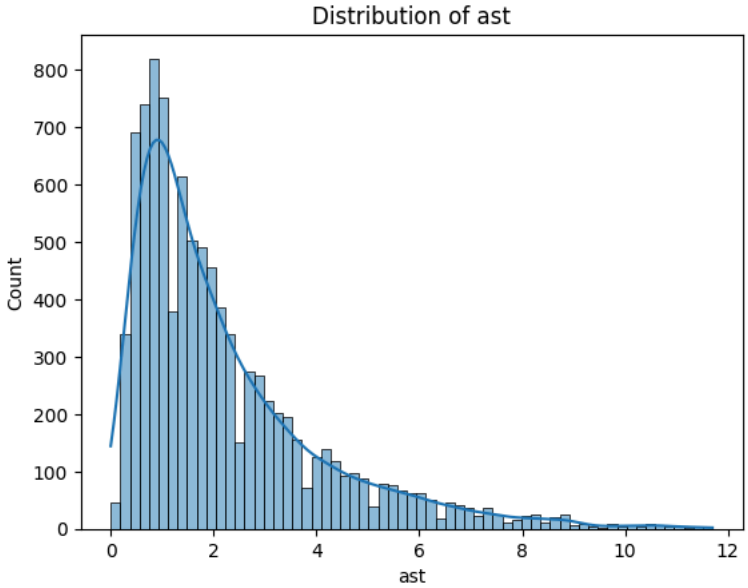

Distribution of pts
Count
pts
0
100
200
300
400
500
600
700
5
10
15
20
25
30
35

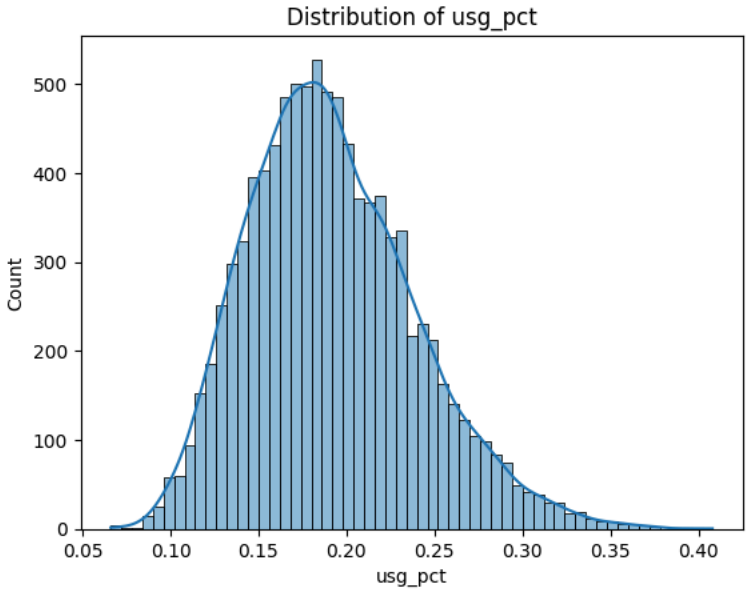

Distribution of gp
Count
gp
0
200
400
600
800
1000
1200
20
30
40
50
60
70
80

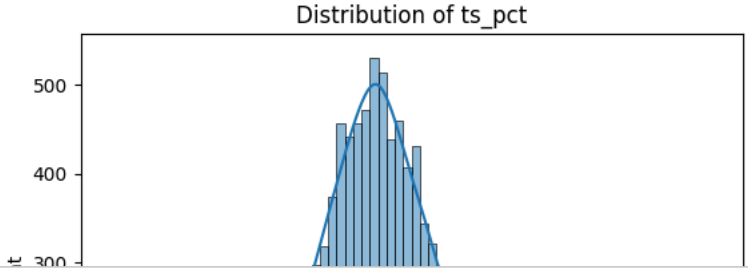

Distribution of age
Count
age
0
200
400
600
800
20
25
30
35
40


Distribution of reb
600

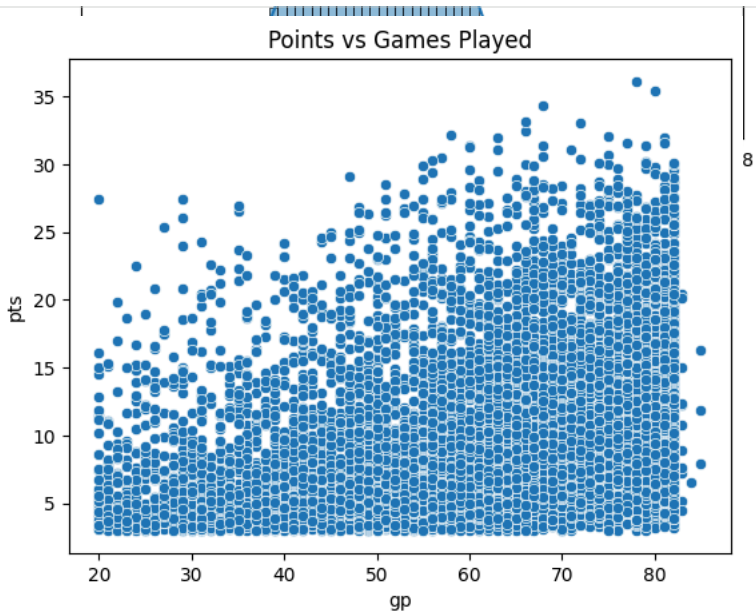

Count
reb


Distribution of ast
Count
ast

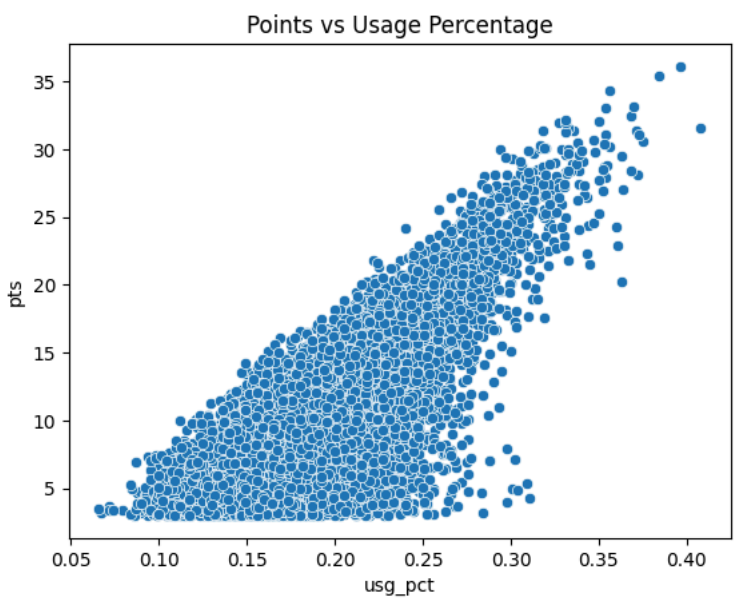

Distribution of usg_pct
Count
usg_pct


Distribution of ts_pct

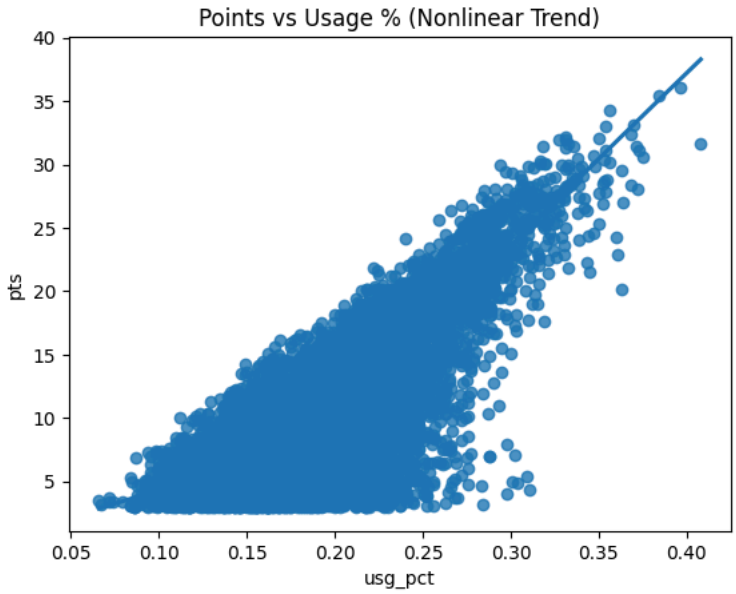
```
plt.figure()
sns.scatterplot(x='gp', y='pts', data=df)
plt.title('Points vs Games Played')
plt.show()
```



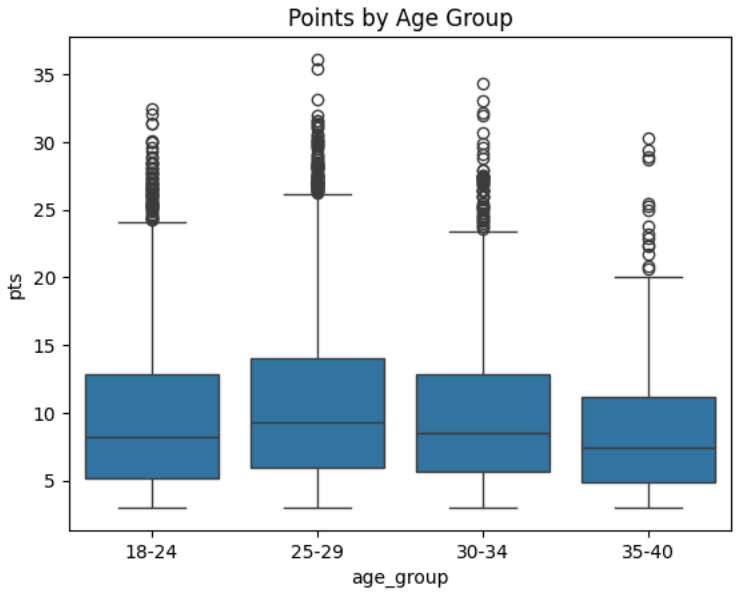

```
plt.figure()
sns.scatterplot(x='usg_pct', y='pts', data=df)
plt.title('Points vs Usage Percentage')
plt.show()
```




```
plt.figure()
sns.regplot(x='usg_pct', y='pts', data=df, lowess=True)
plt.title('Points vs Usage % (Nonlinear Trend)')
plt.show()
```




```
# Create age groups
df['age_group'] = pd.cut(df['age'],
                         bins=[18,24,29,34,40],
                         labels=['18-24','25-29','30-34','35-40'])

plt.figure()
sns.boxplot(x='age_group', y='pts', data=df)
plt.title('Points by Age Group')
plt.show()
```




```
plt.figure(figsize=(10,8))
corr = df.corr(numeric_only=True)
sns.heatmap(corr, cmap='coolwarm')
plt.title('Correlation Heatmap')
plt.show()
```

Exploratory Visualization Findings

After cleaning the dataset by removing players with fewer than 20 games played and fewer than 3 points per game, I explored the distributions and relationships between key variables to better understand the structure of the data before building any regression models.

Distributions

The distribution of points per game is clearly right-skewed. Most players score between about 3 and 12 points, while a much smaller number score above 20. This shows that scoring is not normally distributed and may require transformation later when building regression models.

Games played is concentrated toward higher values, especially between 60 and 82 games. This makes sense because players who remain on rosters for full seasons tend to accumulate more games. The distribution is not perfectly normal but less skewed than points.

Age appears closer to a normal distribution, centered around 25 to 29 years old. This suggests that age may not require heavy transformation, although performance may differ across age groups rather than change linearly.

Rebounds and assists are also right-skewed, similar to points. Most players contribute modestly in these categories, while a small number contribute at very high levels.

Usage percentage looks much closer to a normal distribution, centered around roughly 18 to 22 percent. True shooting percentage also appears approximately normally distributed.

Overall, several performance metrics are skewed, which is important for future modeling assumptions.

Relationships Between Variables

The scatterplot of points versus games played shows a positive relationship. Players who play more games tend to score more, but there is still large variation at every level of games played. This suggests that games played alone is not enough to explain scoring output.

The relationship between points and usage percentage is stronger and more consistent. As usage increases, scoring increases in a clearer upward pattern. This suggests that offensive involvement is likely a key driver of scoring performance.

The pattern does not appear perfectly linear, and there may be some curvature at higher usage levels. This could indicate diminishing returns at very high usage percentages and suggests that nonlinear modeling might improve future regression results.

Interpretation

Based on the visual analysis:

Points, rebounds, and assists are not normally distributed and may require transformation.

Usage percentage appears to be one of the strongest predictors of scoring.

Age may need to be explored categorically rather than strictly continuous.

Scoring is influenced by multiple factors and is unlikely to be explained by a single variable.

These exploratory visualizations help guide future regression modeling decisions and highlight potential nonlinear relationships and distribution concerns.

```
from google.colab import files
uploaded = files.upload()

file_name = list(uploaded.keys())[0]
print("Loaded:", file_name)

import pandas as pd
df = pd.read_csv(file_name)

df.head()
```

Choose Files all_seasons.csv

all_seasons.csv(text/csv) - 1920482 bytes, last modified: 10/13/2023 - 100% done

```
Saving all_seasons.csv to all_seasons.csv
Loaded: all_seasons.csv
```

	Unnamed: 0	player_name	team_abbreviation	age	player_height	player_we
0	0	Randy Livingston	HOU	22.0	193.04	94.80
1	1	Gaylon Nickerson	WAS	28.0	190.50	86.18
2	2	George Lynch	VAN	26.0	203.20	103.41
		George				

```
print(df.columns)
```

```
Index(['Unnamed: 0', 'player_name', 'team_abbreviation', 'age',
       'player_height', 'player_weight', 'college', 'country', 'draft_yea
       'draft_round', 'draft_number', 'gp', 'pts', 'reb', 'ast', 'net_rat
       'oreb_pct', 'dreb_pct', 'usg_pct', 'ts_pct', 'ast_pct', 'season'],
      dtype='object')
```

```
# ========================
# Data Cleaning
# ========================

# remove players with very low playing time / scoring
# professor suggested this to avoid extreme values

df_clean = df[(df["gp"] >= 20) & (df["pts"] >= 3)].copy()

print("Original dataset shape:", df.shape)
print("Cleaned dataset shape:", df_clean.shape)
```

```
df_clean.head()
```

```
Original dataset shape: (12844, 22)
Cleaned dataset shape: (9614, 22)
```

	Unnamed: 0	player_name	team_abbreviation	age	player_height	player_we
0	0	Randy Livingston	HOU	22.0	193.04	94.80
2	2	George Lynch	VAN	26.0	203.20	103.41
3	3	George McCloud	LAL	30.0	203.20	102.05

```
from sklearn.preprocessing import PowerTransformer

vars_to_transform = ["pts", "reb", "ast"]

pt = PowerTransformer(method="yeo-johnson")

df_clean[vars_to_transform] = pt.fit_transform(df_clean[vars_to_transfor

print("Transformation complete")

df_clean.head()
```

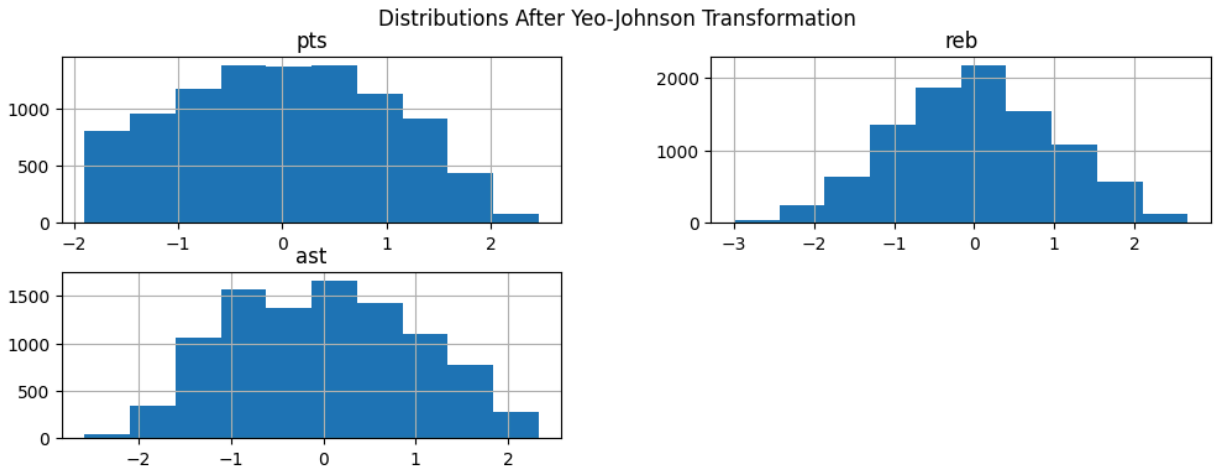
```
Transformation complete
```

	Unnamed: 0	player_name	team_abbreviation	age	player_height	player_we
0	0	Randy Livingston	HOU	22.0	193.04	94.80
2	2	George Lynch	VAN	26.0	203.20	103.41
3	3	George McCloud	LAL	30.0	203.20	102.05
5	5	Gerald Wilkins	ORL	33.0	198.12	102.05
6	6	Gheorghe Muresan	WAS	26.0	231.14	137.43

5 rows × 22 columns

```
import matplotlib.pyplot as plt
```

```
df_clean[vars_to_transform].hist(figsize=(12,4))

plt.suptitle("Distributions After Yeo-Johnson Transformation")

plt.show()
```




```
# ========================
# Grouping Data
# ========================

group_team = df_clean.groupby("team_abbreviation")[["pts","reb","ast"]].

print("Average player performance by team")

display(group_team)
```

Next steps: Generate code with group_team New interactive sheet

```
Average player performance by team
```

team_abbreviation	pts	reb	ast
ATL	-0.045064	-0.063061	-0.061816
BKN	0.011660	-0.026606	-0.011100
BOS	0.005031	0.036930	0.054134
CHA	-0.005248	-0.012435	-0.025078
CHH	-0.110837	-0.062735	-0.006761

```
# ========================
# Aggregation
# ========================

team_summary = df_clean.groupby("team_abbreviation")[["pts","reb","ast"]

print("Summary statistics by team")

display(team_summary)
```

HOU	0.079215	0.083086	-0.003869
IND	0.052006	0.036876	0.032270
LAC	0.027888	-0.043052	-0.005893
LAL	-0.000168	0.072482	0.074462
MEM	-0.024439	-0.018506	-0.015642
MIA	0.030136	0.030362	0.025713
MIL	0.011027	0.053271	0.043205
MIN	0.032813	-0.041083	0.062789
NJN	-0.006640	0.051525	-0.040397
NOH	-0.147328	-0.043666	-0.179176
NOK	0.041388	0.162782	-0.365902
NOP	0.135557	0.115697	0.124429
NYK	0.034357	0.062607	-0.009405
OKC	-0.012429	0.076751	-0.126243
ORL	-0.024806	-0.047936	-0.026182
PHI	0.007798	0.053062	-0.034340
PHX	0.067948	-0.002265	-0.001920
POR	0.003304	0.023731	0.002832

POR	[illegible]	[illegible]	[illegible]
SAC	0.094835	0.021960	0.051278
SAS	-0.072216	-0.095416	0.059374
SEA	-0.079296	-0.009217	-0.134665
TOR	0.018012	0.036442	-0.041289
UTA	-0.045295	-0.116836	0.002057
VAN	-0.319708	-0.174274	-0.064045
WAS	-0.028782	-0.013434	-0.061459

Next steps: Generate code with `team_summary` New interactive sheet

```
Summary statistics by team
```

	pts			reb		
	mean	median	max	mean	median	max
team_abbreviation						
ATL	-0.045064	-0.021116	2.145092	-0.063061	-0.060394	2.467548
BKN	0.011660	-0.053641	2.161344	-0.026606	-0.010303	2.021883
BOS	0.005031	-0.010364	2.172076	0.036930	0.086172	2.005800
CHA	-0.005248	0.093327	1.908773	-0.012435	0.038533	2.231993
CHH	-0.110837	-0.010364	1.983811	-0.062735	-0.274594	2.069216

```
# ========================
# Pivot Table
# ========================

pivot_table = pd.pivot_table(
    df_clean,
    values="pts",
    index="team_abbreviation",
    aggfunc="mean"
)

print("Average points by team")

display(pivot_table)
```

MEM	-0.024439	0.031861	2.019924	-0.018506	-0.060394	2.231993
MIA	0.030136	0.031861	2.177411	0.030362	-0.010303	2.443015
MIL	0.011027	-0.053641	2.224552	0.053271	0.086172	2.379986
MIN	0.032813	0.052594	1.965412	-0.041083	-0.164588	2.573498
NJN	-0.006640	-0.031868	1.882877	0.051525	0.062352	2.379986
NOH	-0.147328	-0.075689	1.724420	-0.043666	-0.060394	2.115232
NOK	0.041388	0.133120	1.346115	0.162782	-0.111800	2.217845
NOP	0.135557	-0.075689	2.060946	0.115697	0.178069	2.287385
NYK	0.034357	0.031861	2.095199	0.062607	-0.010303	2.314388
OKC	-0.012429	-0.190297	2.270173	0.076751	0.222423	2.021883
ORL	-0.024806	0.031861	2.275152	-0.047936	-0.060394	2.491710
PHI	0.007798	0.093327	2.323997	0.053062	0.086172	2.379986
PHX	0.067948	0.103333	2.117589	0.002265	0.086172	2.130290

POR	-0.003304	0.031861	2.280114	0.023731	0.038533	2.367078
SAC	0.094835	0.113339	2.001980	0.021960	-0.010303	2.300943
SAS	-0.072216	-0.075689	1.902342	-0.095416	-0.218827	2.287385
SEA	-0.079296	-0.098019	1.959226	-0.009217	0.132667	1.835830
TOR	0.018012	0.031861	2.031764	0.036442	0.132667	2.084698
UTA	-0.045295	-0.010364	2.019924	-0.116836	-0.164588	2.515511
VAN	-0.319708	-0.499989	1.731674	-0.174274	-0.111800	1.853623
WAS	-0.028782	-0.053641	2.234818	-0.013434	0.086172	2.084698

Next steps: Generate code with `pivot_table` New interactive sheet

```
Average points by team
```

team_abbreviation	pts
ATL	-0.045064
BKN	0.011660
BOS	0.005031
CHA	-0.005248
CHH	-0.110837

```
# ========================
# Cross Tabulation
# ========================

cross_tab = pd.crosstab(df_clean["team_abbreviation"], df_clean["season"

print("Cross tabulation of team vs season")

display(cross_tab)
```

HOU	0.079215
IND	0.052006
LAC	0.027888
LAL	-0.000168
MEM	-0.024439
MIA	0.030136
MIL	0.011027
MIN	0.032813
NJN	-0.006640
NOH	-0.147328
NOK	0.041388
NOP	0.135557
NYK	0.034357
OKC	-0.012429
ORL	-0.024806
PHI	0.007798
PHX	0.067948
POR	0.003304

Data Aggregation and Group Operations

For this part of the analysis, I performed several data aggregation and grouping operations on the cleaned dataset. Before this step, I removed players who played fewer than 20 games and averaged fewer than 3 points per game in order to avoid extremely small sample sizes that could distort the results. After applying this filter, the dataset was reduced from 12,844 observations to 9,614 observations.

Next, I applied a Yeo-Johnson transformation to the points, rebounds, and assists variables. These variables originally showed right-skewed distributions, so the transformation was used to make them more normally distributed. After applying the transformation, the histograms show that the distributions of points, rebounds, and assists are more symmetric, which will help when building regression models later in the project.

I then grouped the data by team using the team_abbreviation variable. This grouping allowed me to calculate the average transformed values of points, rebounds, and assists for players on each team. The grouped results show that there are small differences across teams, with some teams having slightly higher average performance levels than others.

I also calculated summary statistics using aggregation functions. Specifically, I computed the mean, median, and maximum values for points, rebounds, and assists by team. These statistics provide a clearer picture of how player performance varies within each team and allow comparisons across teams.

Next, I created a pivot table to summarize the average points scored by players for each team. Pivot tables provide a structured way to summarize relationships between categorical variables and numerical variables and make it easier to compare performance across groups.

Finally, I created a cross tabulation between team and season. This cross tabulation shows how player observations are distributed across teams and seasons and helps describe the structure of the dataset.

Overall, these grouping and aggregation operations helped summarize the dataset and provided additional insight into how player performance varies across teams and seasons. These results will help guide the next stage of the project, which will involve building regression models to analyze relationships between the variables.

```
# ==========================================
# FINAL PROJECT ANALYSIS NOTEBOOK
# NBA PLAYER SCORING PROJECT
# ==========================================

# STEP 1: Upload file
from google.colab import files
uploaded = files.upload()
```

Choose Files all_seasons.csv

all_seasons.csv(text/csv) - 1920482 bytes, last modified: 10/13/2023 - 100% done

```
Saving all_seasons.csv to all_seasons.csv
```

```
import pandas as pd

df = pd.read_csv("all_seasons.csv")

df.head()
```

	Unnamed: 0	player_name	team_abbreviation	age	player_height	player_weight	college	country	dra
0	0	Randy Livingston	HOU	22.0	193.04	94.800728	Louisiana State	USA	
1	1	Gaylon Nickerson	WAS	28.0	190.50	86.182480	Northwestern Oklahoma	USA	
2	2	George Lynch	VAN	26.0	203.20	103.418976	North Carolina	USA	
3	3	George McCloud	LAL	30.0	203.20	102.058200	Florida State	USA	
4	4	George Zidek	DEN	23.0	213.36	119.748288	UCLA	USA	

5 rows × 22 columns

```
df.columns
```

```
Index(['Unnamed: 0', 'player_name', 'team_abbreviation', 'age',
       'player_height', 'player_weight', 'college', 'country', 'draft_year',
       'draft_round', 'draft_number', 'gp', 'pts', 'reb', 'ast', 'net_rating',
       'oreb_pct', 'dreb_pct', 'usg_pct', 'ts_pct', 'ast_pct', 'season'],
      dtype='object')
```

```
df['ppg'] = df['pts'] / df['gp']

df[['player_name','pts','gp','ppg']].head()
```

	player_name	pts	gp	ppg
0	Randy Livingston	3.9	64	0.060937
1	Gaylon Nickerson	3.8	4	0.950000
2	George Lynch	8.3	41	0.202439
3	George McCloud	10.2	64	0.159375
4	George Zidek	2.8	52	0.053846

```
import pandas as pd

df = pd.read_csv("all_seasons.csv")
df['ppg'] = df['pts']
df = df[df['gp'] >= 20]

df[['player_name','gp','pts','ppg','reb','ast']].head()
```

	player_name	gp	pts	ppg	reb	ast
0	Randy Livingston	64	3.9	3.9	1.5	2.4
2	George Lynch	41	8.3	8.3	6.4	1.9
3	George McCloud	64	10.2	10.2	2.8	1.7
4	George Zidek	52	2.8	2.8	1.7	0.3
5	Gerald Wilkins	80	10.6	10.6	2.2	2.2

```
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import r2_score

analysis_df = df[['ppg','reb','ast','age','net_rating']].dropna().copy()

print("Rows used:", analysis_df.shape[0])
```

```
print("\nSummary stats:")
print(analysis_df.describe().round(2))

X = analysis_df[['reb','ast','age','net_rating']]
y = analysis_df['ppg']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

model = LinearRegression()
model.fit(X_train, y_train)

pred = model.predict(X_test)

print("\nR2 Score:", round(r2_score(y_test, pred), 3))
print("\nCoefficients:")
for col, coef in zip(X.columns, model.coef_):
    print(f"{col}: {coef:.3f}")
```

```
Rows used: 10720

Summary stats:
            ppg       reb       ast       age  net_rating
count  10720.00  10720.00  10720.00  10720.00    10720.00
mean       9.22      3.93      2.04     27.17       -1.09
std        5.93      2.45      1.85      4.33        6.44
min        0.30      0.10      0.00     18.00      -40.00
25%        4.70      2.10      0.80     24.00       -5.30
50%        7.80      3.30      1.40     27.00       -0.80
75%       12.50      5.10      2.70     30.00        3.20
max       36.10     16.10     11.70     43.00       19.50

R2 Score: 0.64

Coefficients:
reb: 1.119
ast: 1.783
age: -0.102
net_rating: 0.055
```

```
from scipy.stats import f_oneway
```

```
anova_df = df[['pts', 'draft_round']].dropna().copy()
anova_df['draft_round'] = anova_df['draft_round'].astype(str)

group1 = anova_df[anova_df['draft_round'] == '1']['pts']
group2 = anova_df[anova_df['draft_round'] == '2']['pts']
group3 = anova_df[~anova_df['draft_round'].isin(['1', '2'])]['pts']

print("Round 1 players:", len(group1))
print("Round 2 players:", len(group2))
print("Other/undrafted players:", len(group3))

f_stat, p_value = f_oneway(group1, group2, group3)

print("\nANOVA F-statistic:", round(f_stat, 3))
print("ANOVA p-value:", round(p_value, 5))
```

```
Round 1 players: 6734
Round 2 players: 2417
Other/undrafted players: 1569

ANOVA F-statistic: 662.188
ANOVA p-value: 0.0
```

```
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import r2_score
from scipy.stats import f_oneway

analysis_df = df[['ppg','reb','ast','age','net_rating']].dropna().copy()
X = analysis_df[['reb','ast','age','net_rating']]
y = analysis_df['ppg']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
model = LinearRegression()
model.fit(X_train, y_train)
pred = model.predict(X_test)

anova_df = df[['pts','draft_round']].dropna().copy()
anova_df['draft_round'] = anova_df['draft_round'].astype(str)
```

```
anova_df['draft_round'] = anova_df['draft_round'].astype(str)

group1 = anova_df[anova_df['draft_round'] == '1']['pts']
group2 = anova_df[anova_df['draft_round'] == '2']['pts']
group3 = anova_df[~anova_df['draft_round'].isin(['1', '2'])]['pts']

f_stat, p_value = f_oneway(group1, group2, group3)

print("R2 Score:", round(r2_score(y_test, pred), 3))
print("ANOVA p-value:", round(p_value, 5))
```

```
R2 Score: 0.64
ANOVA p-value: 0.0
```

Final Results Comment

Using 10,720 player seasons with at least 20 games played, the linear regression model showed an R2 of 0.64. This means rebounds, assists, age, and net rating explain about 64% of the variation in points per game, which indicates a moderately strong model for this project. The ANOVA test comparing scoring across draft groups was also statistically significant ($F = 662.188$, $p < 0.001$), showing that points per game differ meaningfully across Round 1, Round 2, and other or undrafted players. Overall, the results suggest that both on court performance variables and draft background are important for understanding NBA scoring output.